

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА №7

З дисципліни «Інфраструктура Інформаційних Технологій»

Тема: «ПОДІЙНО-ОРІЄНТОВАНА АРХІТЕКТУРА.
RABBITMQ»

Виконала:

студентка групи ІА-32

Павлюк Вероніка Олександрівна

Перевірив:

Цимбал С. І.

Тема: Подійно-орієнтована архітектура. RABBITMQ

Мета роботи: Ознайомитися з подійно-орієнтованою архітектурою та протоколом AMQP. Опанувати використання RabbitMQ для пересилання повідомлень між компонентами системи з подійно-орієнтованою архітектурою.

Хід роботи

1. Запустити RabbitMQ у Docker контейнері та перевірити доступність через вебінтерфейс (<http://localhost:15672>).

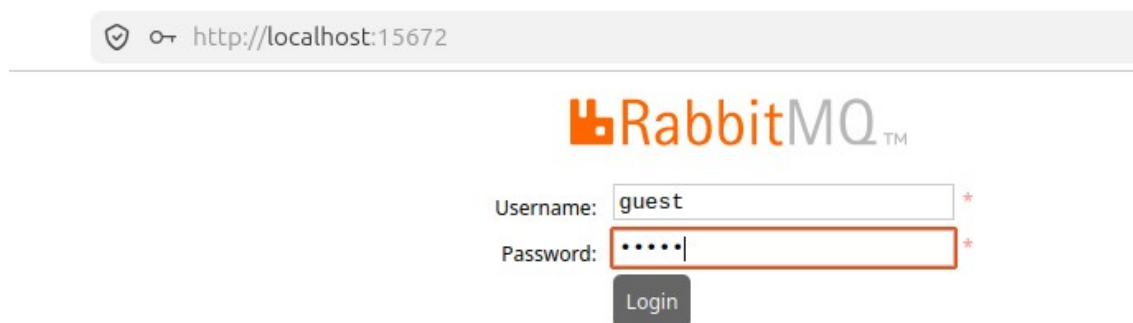


Рисунок 1 - Веб-інтерфейс RabbitMQ

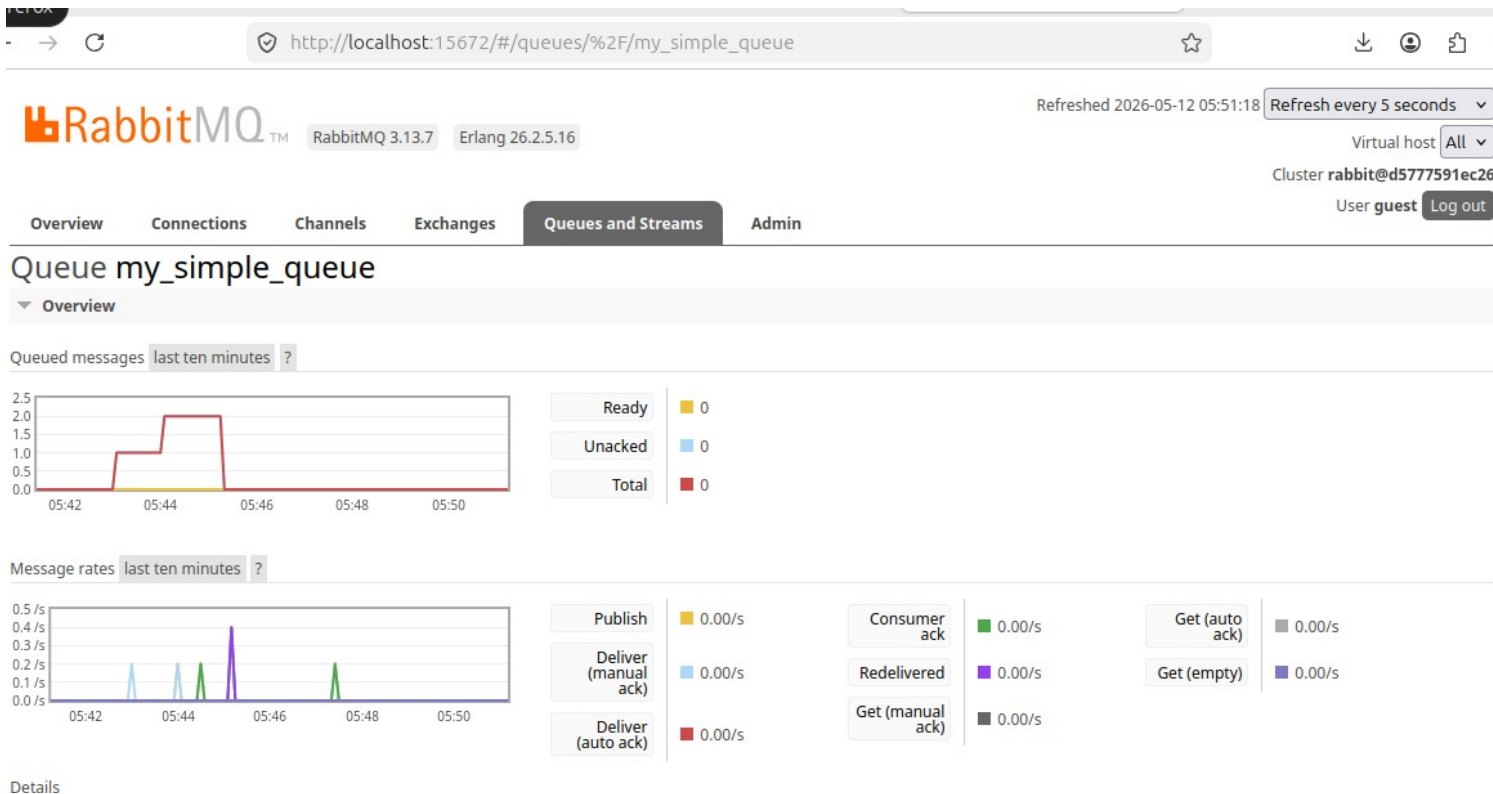
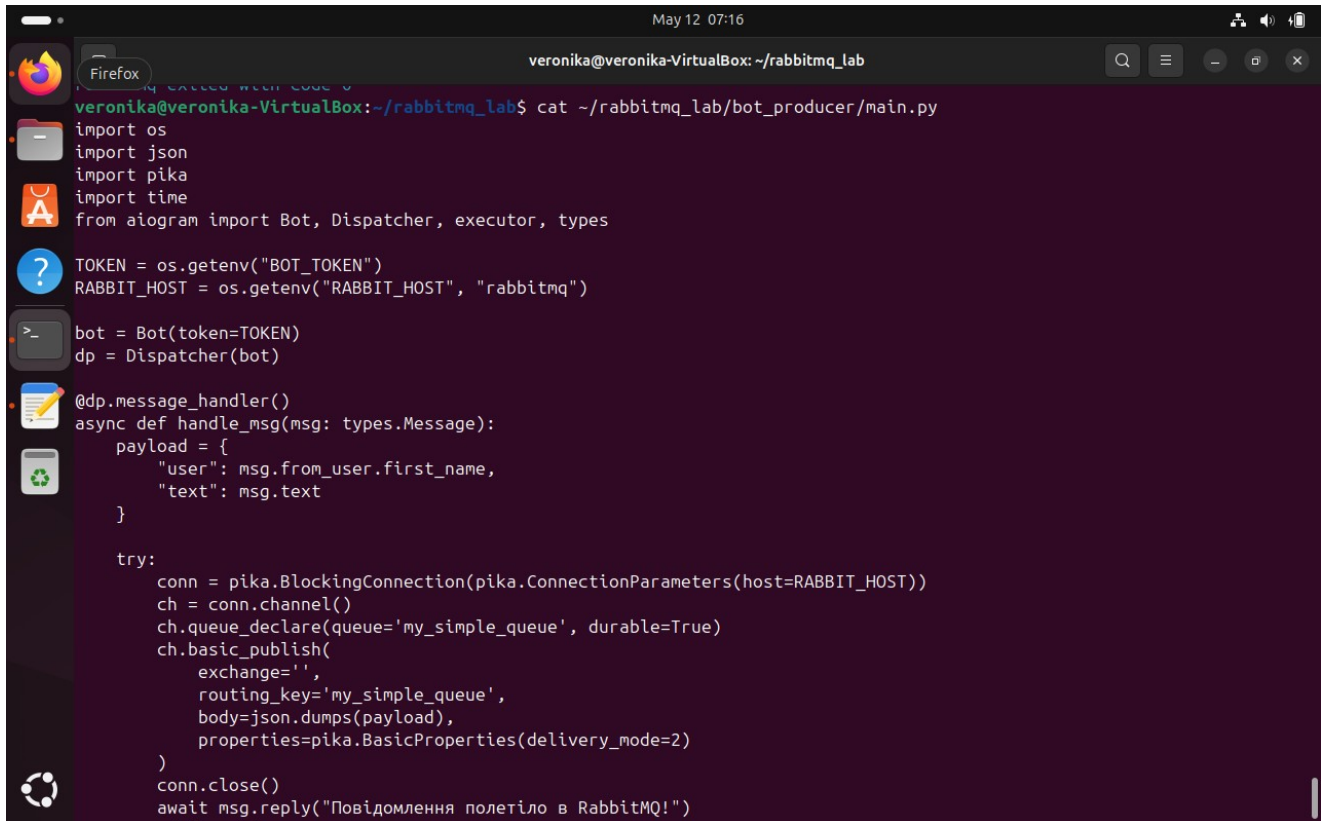


Рисунок 2 - Веб-інтерфейс RabbitMQ (Загальний вигляд черги)

2. Реалізувати двох ботів (Telegram/Discord). Перший з них у відповідь на певні команди користувача надсилатиме повідомлення в чергу, другий — вичитує повідомлення з черги та, за наявності нових, надсилає їх користувачу.



```
veronika@veronika-VirtualBox: ~/rabbitmq_lab
cat ~/rabbitmq_lab/bot_producer/main.py
import os
import json
import pika
import time
from aiogram import Bot, Dispatcher, executor, types

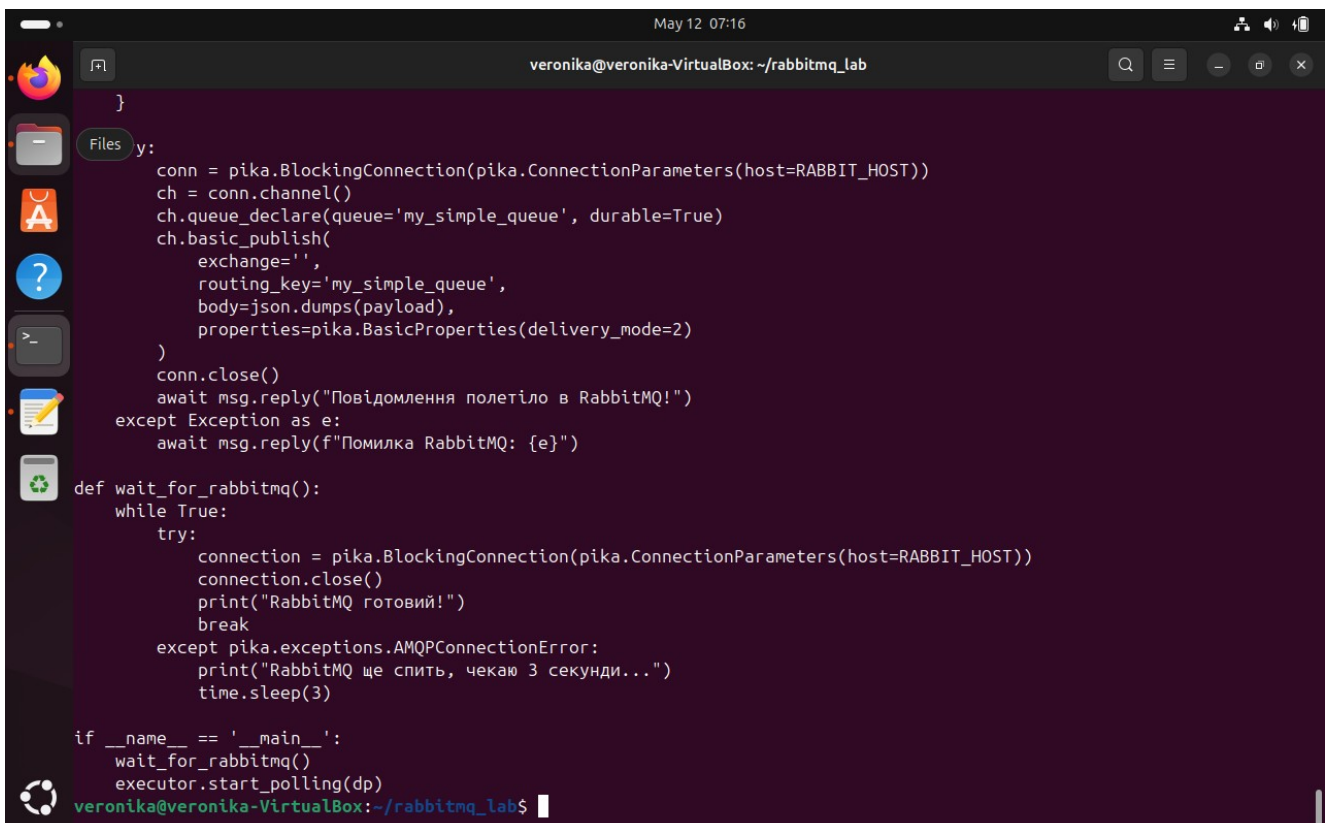
TOKEN = os.getenv("BOT_TOKEN")
RABBIT_HOST = os.getenv("RABBIT_HOST", "rabbitmq")

bot = Bot(token=TOKEN)
dp = Dispatcher(bot)

@dp.message_handler()
async def handle_msg(msg: types.Message):
    payload = {
        "user": msg.from_user.first_name,
        "text": msg.text
    }

    try:
        conn = pika.BlockingConnection(pika.ConnectionParameters(host=RABBIT_HOST))
        ch = conn.channel()
        ch.queue_declare(queue='my_simple_queue', durable=True)
        ch.basic_publish(
            exchange='',
            routing_key='my_simple_queue',
            body=json.dumps(payload),
            properties=pika.BasicProperties(delivery_mode=2)
        )
        conn.close()
    except:
        await msg.reply("Повідомлення полетіло в RabbitMQ!")
```

Рисунок 3 - Код для Бота-Продюсера



```
veronika@veronika-VirtualBox: ~/rabbitmq_lab
cat ~/rabbitmq_lab/bot_producer/main.py
}
y:
    conn = pika.BlockingConnection(pika.ConnectionParameters(host=RABBIT_HOST))
    ch = conn.channel()
    ch.queue_declare(queue='my_simple_queue', durable=True)
    ch.basic_publish(
        exchange='',
        routing_key='my_simple_queue',
        body=json.dumps(payload),
        properties=pika.BasicProperties(delivery_mode=2)
    )
    conn.close()
    await msg.reply("Повідомлення полетіло в RabbitMQ!")
except Exception as e:
    await msg.reply(f"Помилка RabbitMQ: {e}")

def wait_for_rabbitmq():
    while True:
        try:
            connection = pika.BlockingConnection(pika.ConnectionParameters(host=RABBIT_HOST))
            connection.close()
            print("RabbitMQ готовий!")
            break
        except pika.exceptions.AMQPConnectionError:
            print("RabbitMQ ще спить, чекаю 3 секунди...")
            time.sleep(3)

if __name__ == '__main__':
    wait_for_rabbitmq()
    executor.start_polling(dp)
veronika@veronika-VirtualBox: ~/rabbitmq_lab$
```

Рисунок 4 - Код для Бота-Продюсера

```
May 12 07:18
veronika@veronika-VirtualBox: ~/rabbitmq_lab
veronika@veronika-VirtualBox:~/rabbitmq_lab$ cat ~/rabbitmq_lab/bot_consumer/main.py
import os
import json
import pika
import time
import asyncio
from aiogram import Bot

TOKEN = os.getenv("BOT_TOKEN")
RABBIT_HOST = os.getenv("RABBIT_HOST", "rabbitmq")
MY_ID = 700350655

bot = Bot(token=TOKEN)

def callback(ch, method, properties, body):
    data = json.loads(body)
    text = f"Нове повідомлення від {data.get('user', 'Анонім')}: {data.get('text', '')}"

    loop = asyncio.get_event_loop()
    loop.run_until_complete(bot.send_message(MY_ID, text))
    ch.basic_ack(delivery_tag=method.delivery_tag)

def main():
    while True:
        try:
            conn = pika.BlockingConnection(pika.ConnectionParameters(host=RABBIT_HOST))
            ch = conn.channel()
            ch.queue_declare(queue='my_simple_queue', durable=True)
            ch.basic_consume(queue='my_simple_queue', on_message_callback=callback)
            print("Consumer підключився до RabbitMQ")
            ch.start_consuming()
        except pika.exceptions.AMQPConnectionError:
            print("Помилка підключення до RabbitMQ")
```

Рисунок 5 - Код для Бота-Консьюмера

```
May 12 07:18
veronika@veronika-VirtualBox: ~/rabbitmq_lab
RABBIT_HOST = os.getenv("RABBIT_HOST", "rabbitmq")
MY_ID = 700350655

bot = Bot(token=TOKEN)

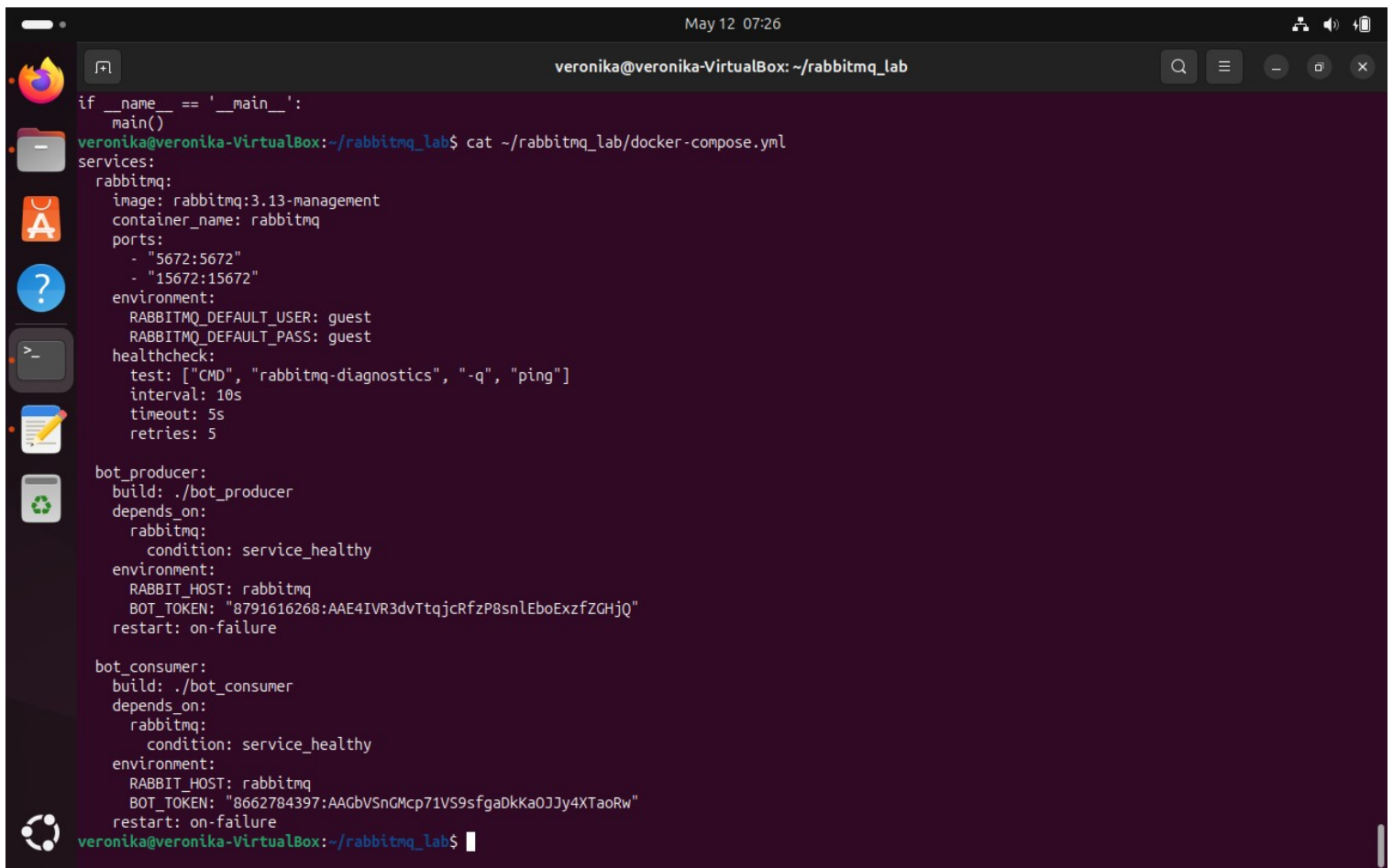
def callback(ch, method, properties, body):
    data = json.loads(body)
    text = f"Нове повідомлення від {data.get('user', 'Анонім')}: {data.get('text', '')}"

    loop = asyncio.get_event_loop()
    loop.run_until_complete(bot.send_message(MY_ID, text))
    ch.basic_ack(delivery_tag=method.delivery_tag)

def main():
    while True:
        try:
            conn = pika.BlockingConnection(pika.ConnectionParameters(host=RABBIT_HOST))
            ch = conn.channel()
            ch.queue_declare(queue='my_simple_queue', durable=True)
            ch.basic_consume(queue='my_simple_queue', on_message_callback=callback)
            print("Consumer підключився до RabbitMQ")
            ch.start_consuming()
        except pika.exceptions.AMQPConnectionError:
            print("Очікування RabbitMQ...")
            time.sleep(3)
        except Exception as e:
            print(f"Помилка: {e}")
            time.sleep(3)

if __name__ == '__main__':
    main()
veronika@veronika-VirtualBox:~/rabbitmq_lab$
```

Рисунок 6 - Код для Бота-Консьюмера



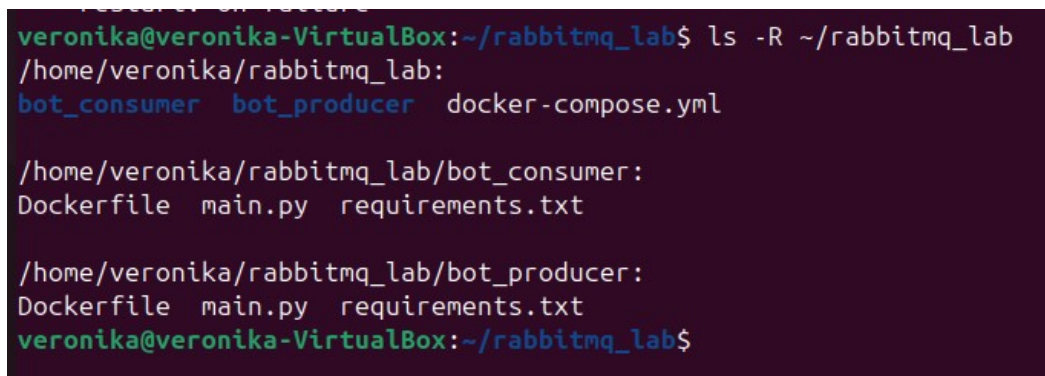
The screenshot shows a terminal window titled 'veronika@veronika-VirtualBox: ~/rabbitmq_lab'. The user has executed the command 'cat ~/rabbitmq_lab/docker-compose.yml'. The output displays the configuration for a Docker Compose project. It defines three services: 'rabbitmq', 'bot_producer', and 'bot_consumer'. The 'rabbitmq' service uses the 'rabbitmq:3.13-management' image and maps ports 5672 and 15672. It sets environment variables for 'RABBITMQ_DEFAULT_USER' and 'RABBITMQ_DEFAULT_PASS' to 'guest'. A healthcheck is configured to run 'rabbitmq-diagnostics -q' every 10 seconds. The 'bot_producer' service is built from the local 'bot_producer' directory and depends on the 'rabbitmq' service being healthy. It sets 'RABBIT_HOST' to 'rabbitmq' and 'BOT_TOKEN' to a specific value. The 'bot_consumer' service is built from the local 'bot_consumer' directory and also depends on the 'rabbitmq' service being healthy, with similar environment variable settings.

```
if __name__ == '__main__':
    main()
veronika@veronika-VirtualBox:~/rabbitmq_lab$ cat ~/rabbitmq_lab/docker-compose.yml
services:
  rabbitmq:
    image: rabbitmq:3.13-management
    container_name: rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  bot_producer:
    build: ./bot_producer
    depends_on:
      rabbitmq:
        condition: service_healthy
    environment:
      RABBIT_HOST: rabbitmq
      BOT_TOKEN: "8791616268:AAE4IVR3dvTtqjcRfzP8snLEboExzfZGHjQ"
    restart: on-failure

  bot_consumer:
    build: ./bot_consumer
    depends_on:
      rabbitmq:
        condition: service_healthy
    environment:
      RABBIT_HOST: rabbitmq
      BOT_TOKEN: "8662784397:AAGbVSnGMcp71VS9sfgaDkKa0JJy4XTaoRw"
    restart: on-failure
veronika@veronika-VirtualBox:~/rabbitmq_lab$
```

Рисунок 7 — Вміст файлу Docker-compose.yml



The screenshot shows a terminal window with the command 'ls -R ~/rabbitmq_lab' executed. The output displays a recursive directory listing of the project. It shows the root directory containing 'bot_consumer', 'bot_producer', and 'docker-compose.yml'. Below these, it shows the contents of the 'bot_consumer' and 'bot_producer' directories, which both contain 'Dockerfile', 'main.py', and 'requirements.txt'.

```
veronika@veronika-VirtualBox:~/rabbitmq_lab$ ls -R ~/rabbitmq_lab
/home/veronika/rabbitmq_lab:
bot_consumer  bot_producer  docker-compose.yml

/home/veronika/rabbitmq_lab/bot_consumer:
Dockerfile  main.py  requirements.txt

/home/veronika/rabbitmq_lab/bot_producer:
Dockerfile  main.py  requirements.txt
veronika@veronika-VirtualBox:~/rabbitmq_lab$
```

Рисунок 8 - Дерево всіх папок і файлів у проекті



Рисунок 9 - Робота Бота-Продюсера (Telegram)

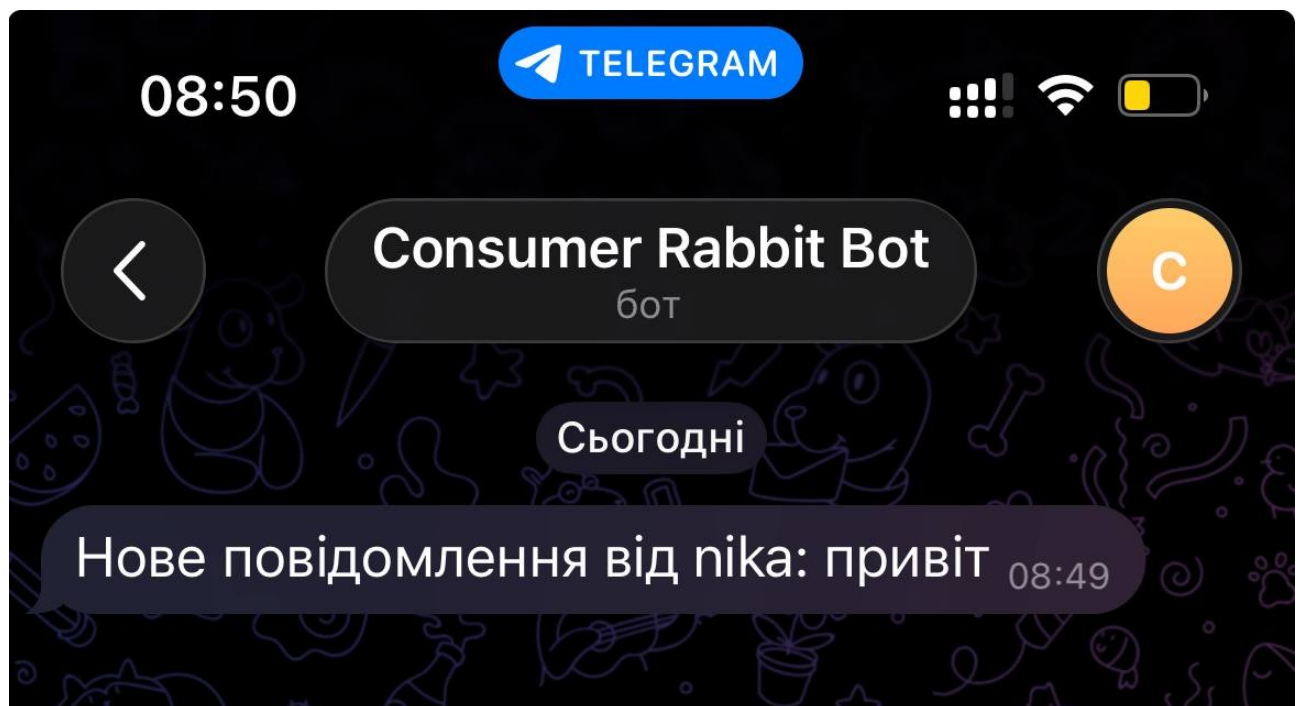


Рисунок 10 - Робота Бота-Консьюмера (Telegram)

Висновок: У ході виконання лабораторної роботи було детально вивчено принципи побудови подійно-орієнтованої архітектури (EDA) та практичні аспекти роботи з брокером повідомлень RabbitMQ. За допомогою Docker Compose було розгорнуто комплексну інфраструктуру, що складається з трьох незалежних сервісів (RabbitMQ та два Python-боти). Реалізовано асинхронний обмін даними між мікросервісами. Використання черг дозволило розірвати прямий зв'язок між відправником (Producer) та отримувачем (Consumer), що підвищує загальну відмовостійкість системи.